

Anexo - Modelo de arquitectura C4



”Porque el club es de los socios, y la gestión es de SocioUnido”

Plataforma inteligente que centraliza la gestión administrativa de los clubes, optimizando el uso de espacios compartidos, previniendo la morosidad y asegurando los accesos al estadio.

Alumno	Correo electrónico	Padrón
Ascencio, Felipe Santino	fascencio@fi.uba.ar	110675
Ghosn, Lautaro Gabriel	lghosn@fi.uba.ar	106998
Guerrero, Martín	mguerrero@fi.uba.ar	107774
Zielonka, Axel	azielonka@fi.uba.ar	110310

Jefe de Cátedra Mg. Ing. Carlos Fontela cfontela@fi.uba.ar

Tutor Mg. Ing. Gabriel Piñeiro gpineiro@fi.uba.ar

Cotutor Ing. Agustín Perrotta aperrotta@fi.uba.ar

Índice

1. Modelo de arquitectura C4	3
1.1. Nivel 1 del modelo de arquitectura C4	3
1.2. Nivel 2 del modelo de arquitectura C4	4
1.3. Nivel 3 del modelo de arquitectura C4	5
1.3.1. Microservicio central (Core)	5
1.3.2. Microservicio de procesamiento de lenguaje natural (PLN)	6
1.3.3. Microservicio de pagos	7
1.3.4. Microservicio de acceso inteligente	8
1.3.5. Microservicio de autenticación	9
1.3.6. Microservicio de analítica e inteligencia artificial	10
1.3.7. Gateway	11
1.3.8. Web administrativa (Panel de gestión)	12
1.3.9. App del Socio (PWA)	13
1.4. Monitoreo y Resiliencia: Servicio de Healthchecker	14
1.4.1. Nivel 3 del modelo de arquitectura C4	15
1.5. Observabilidad y centralización de logs: Integración con Datadog	16
1.5.1. Ejes fundamentales de la implementación y casos de uso	16

1. Modelo de arquitectura C4

En el presente anexo se documenta la arquitectura de software de la plataforma **SocioUnido** utilizando el modelo C4. Este modelo permite visualizar el sistema desde distintos niveles de abstracción, facilitando la comprensión tanto para perfiles técnicos de desarrollo y operaciones, como para perfiles de negocio.

A continuación, se presentan los diagramas correspondientes a los niveles de Contexto (Nivel 1), Contenedores (Nivel 2) y Componentes (Nivel 3) para los distintos microservicios que componen el ecosistema. Para el nivel de componentes, se expone en cada caso el diagrama técnico (base) seguido inmediatamente por su versión visual y simplificada.

1.1. Nivel 1 del modelo de arquitectura C4

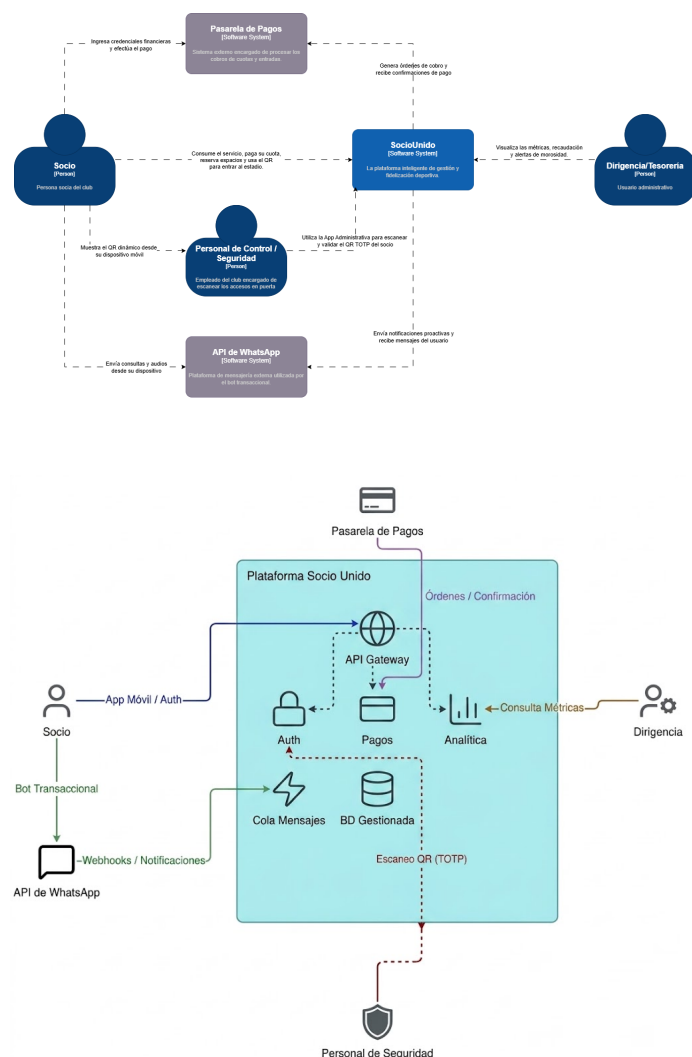


Figura 1: Diagrama C4 nivel 1: Contexto del sistema.

Arriba: Vista técnica base.

Abajo: Vista visual simplificada. Interacción de la plataforma SocioUnido con los distintos actores (Socios, Dirigencia, Personal de Control) y sistemas externos (Pasarela de Pagos, Interfaz de Programación de Aplicaciones [API] de WhatsApp).

1.2. Nivel 2 del modelo de arquitectura C4

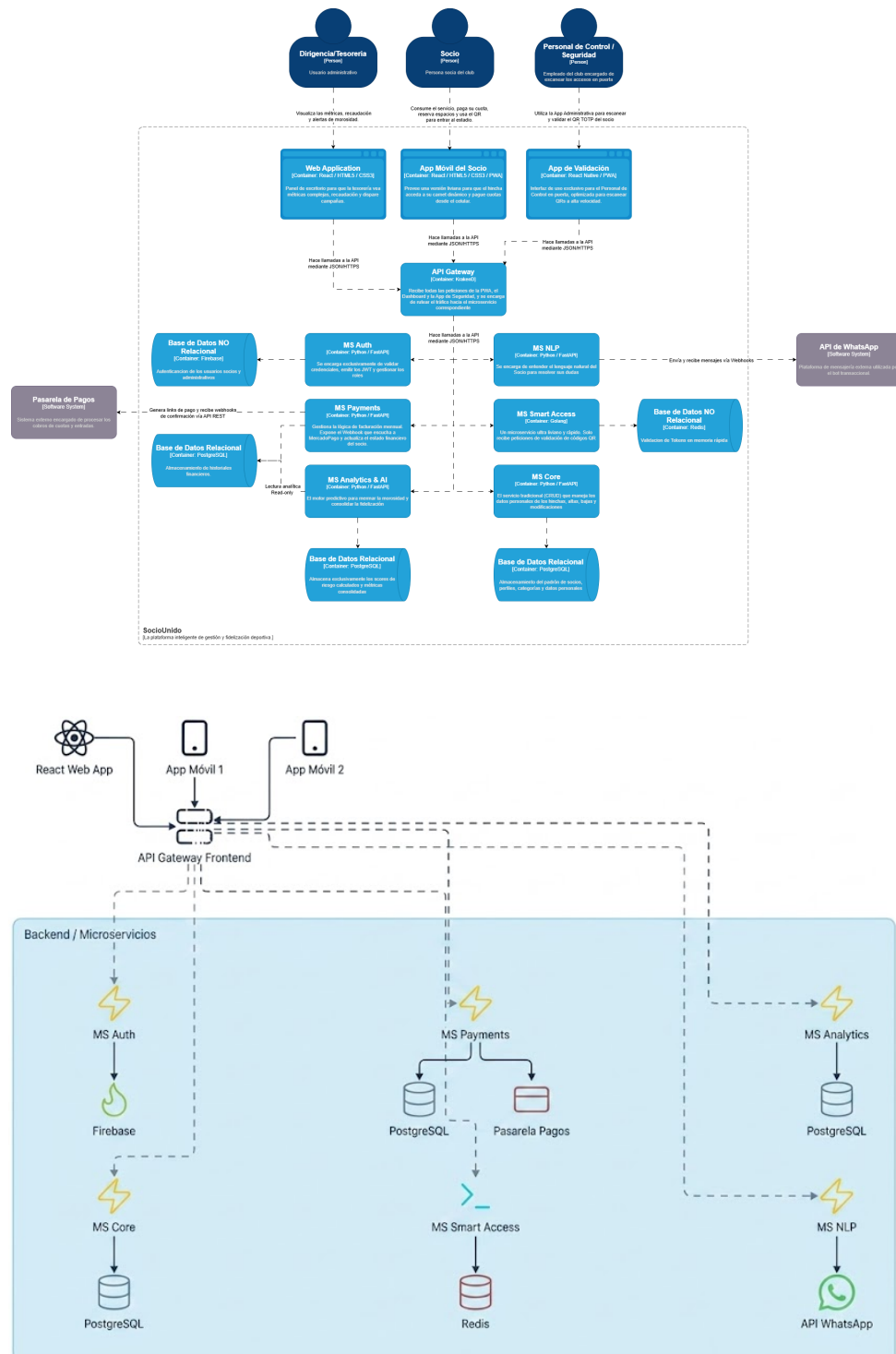


Figura 2: Diagrama C4 nivel 2: Contenedores.

Arriba: Vista técnica base.

Abajo: Vista visual simplificada. Vista de alto nivel de las aplicaciones web, móviles, bases de datos y la arquitectura basada en microservicios detrás de la puerta de enlace de API (API Gateway).

1.3. Nivel 3 del modelo de arquitectura C4

1.3.1. Microservicio central (Core)

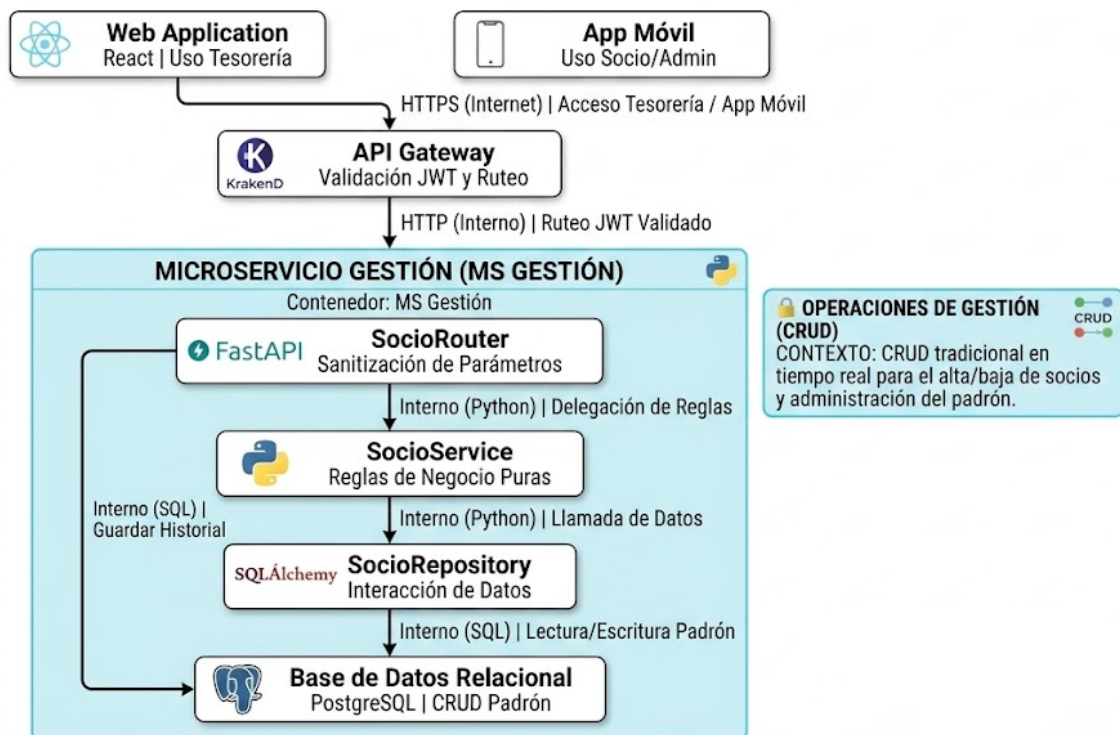
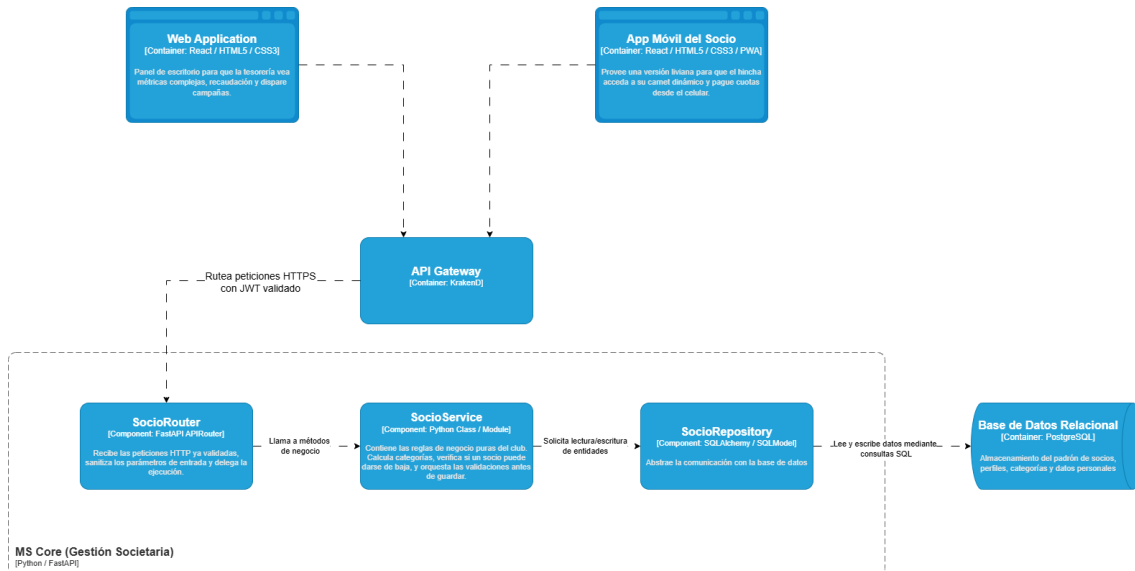


Figura 3: Diagrama C4 nivel 3: Componentes (Microservicio de gestión societaria).

Arriba: Vista técnica base.

Abajo: Vista visual simplificada. Detalle del microservicio principal encargado del CRUD de usuarios, perfiles y reglas de negocio del padrón societario.

1.3.2. Microservicio de procesamiento de lenguaje natural (PLN)

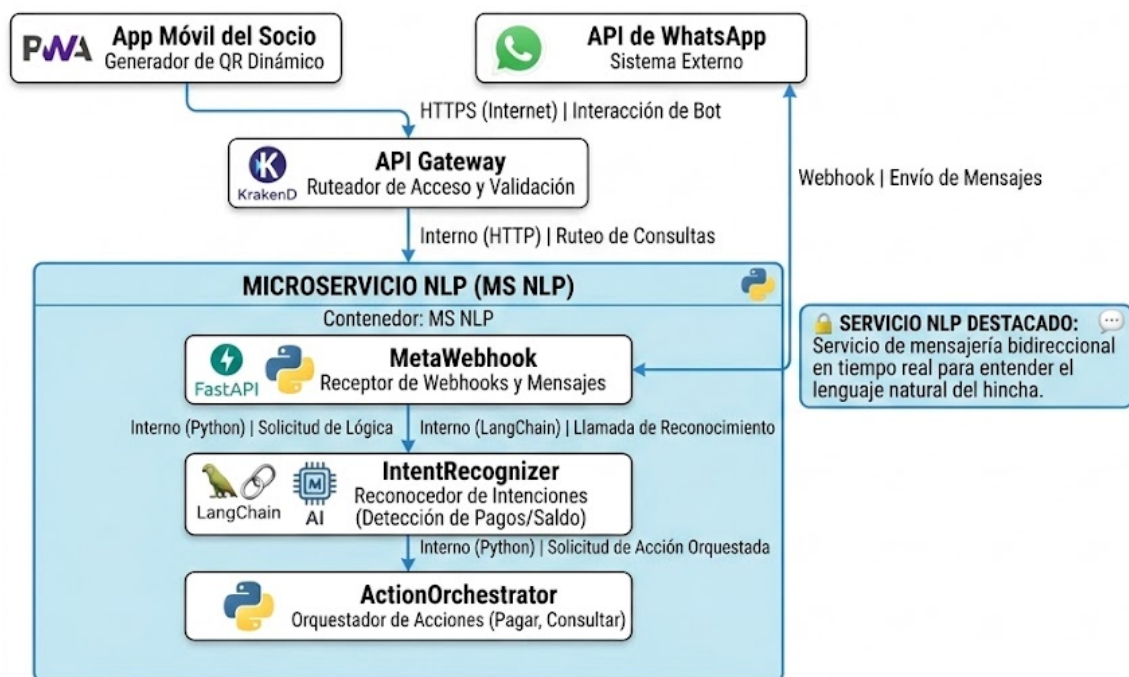
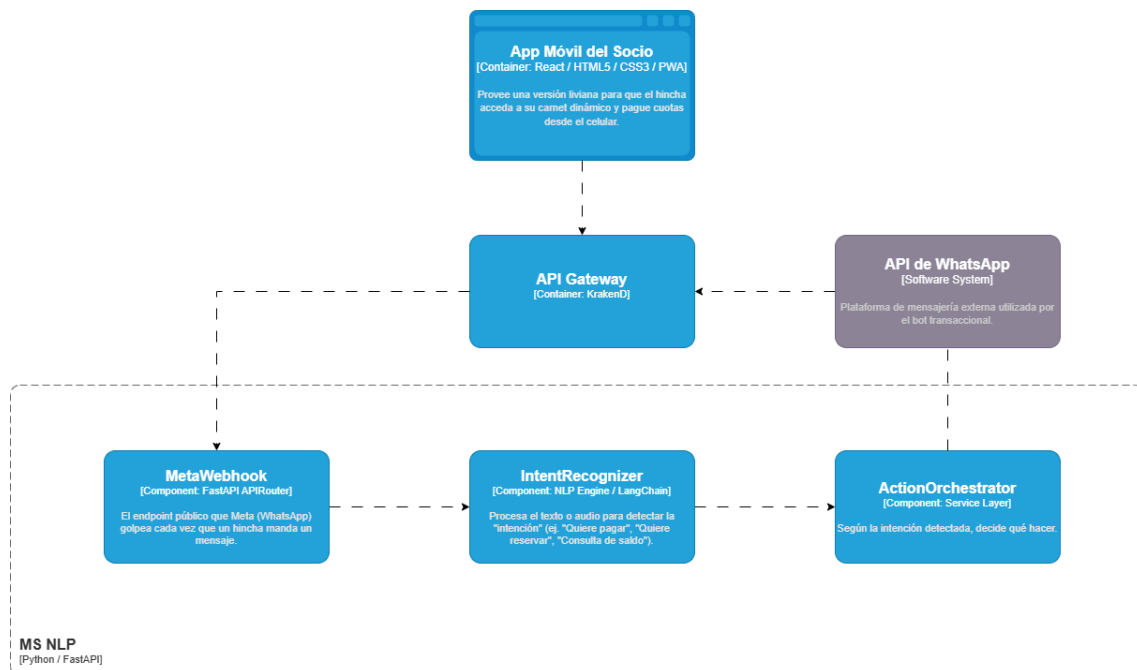


Figura 4: Diagrama C4 nivel 3: Componentes (Microservicio de Procesamiento de Lenguaje Natural [PLN]).

Arriba: Vista técnica base.

Abajo: Vista visual simplificada. Detalle del motor integrado vía Webhook con la API de WhatsApp.

1.3.4. Microservicio de acceso inteligente

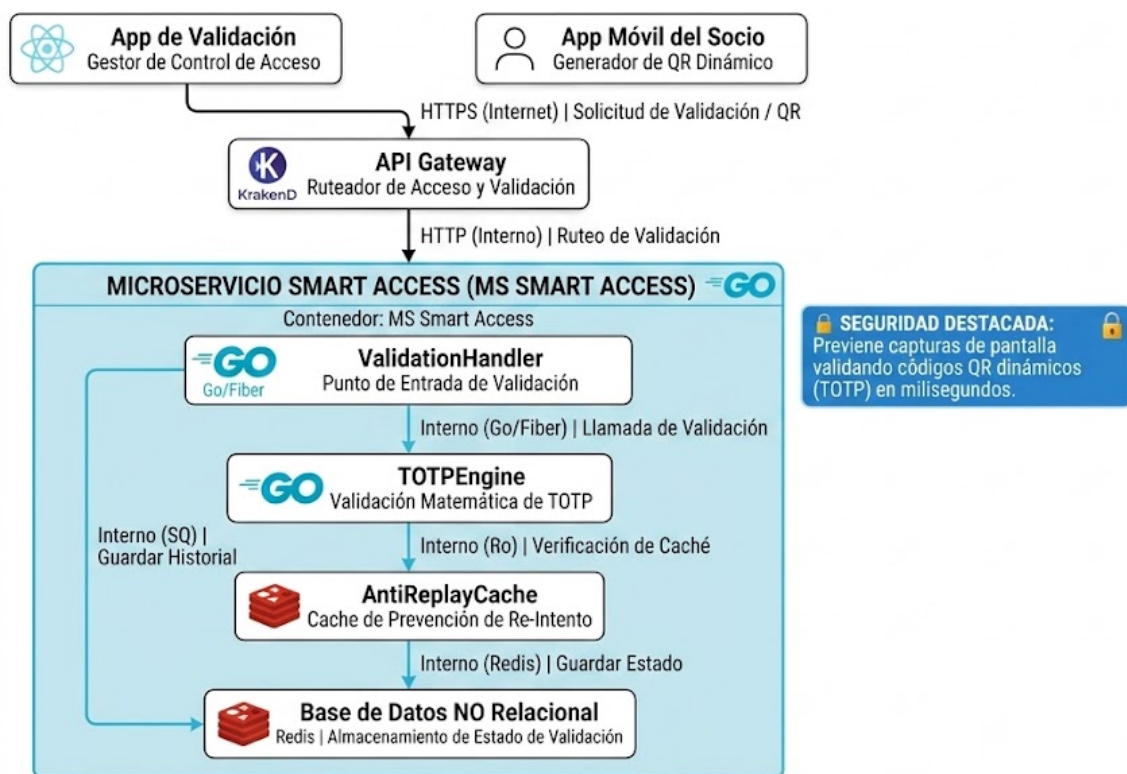
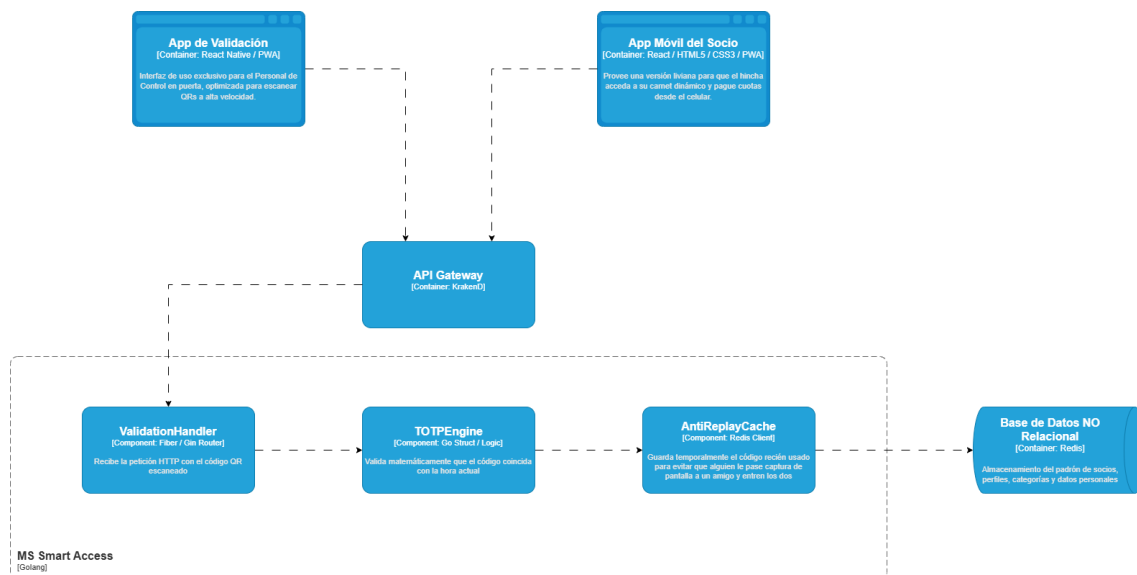


Figura 6: Diagrama C4 nivel 3: Componentes (Microservicio de acceso inteligente).

Arriba: Vista técnica base.

Abajo: Vista visual simplificada. Lógica interna de validación mediante TOTP para estadios.

1.3.5. Microservicio de autenticación

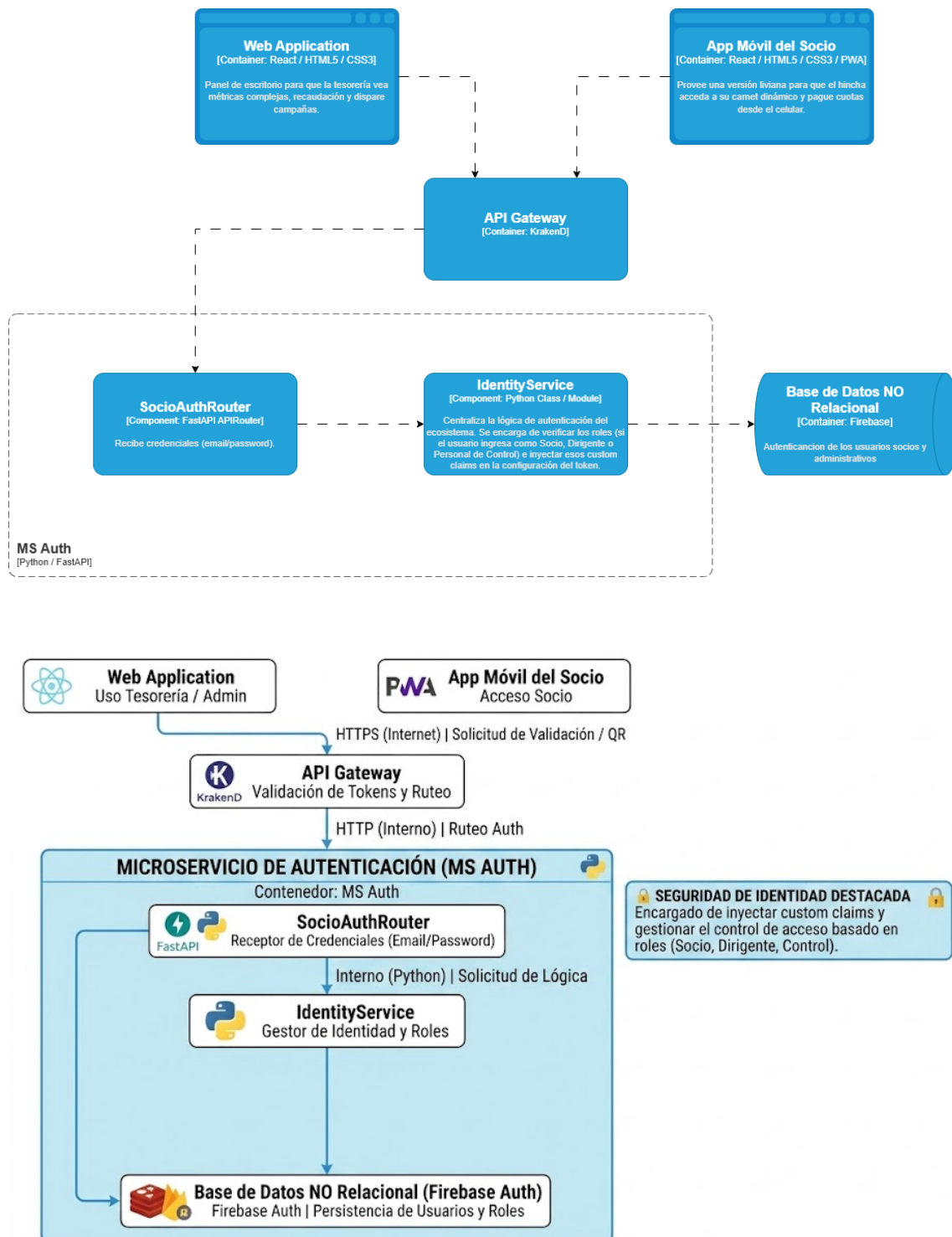


Figura 7: Diagrama C4 nivel 3: Componentes (Microservicio de autenticación).

Arriba: Vista técnica base.

Abajo: Vista visual simplificada. Sistema para el manejo de identidades y generación de JWT.

1.3.6. Microservicio de analítica e inteligencia artificial

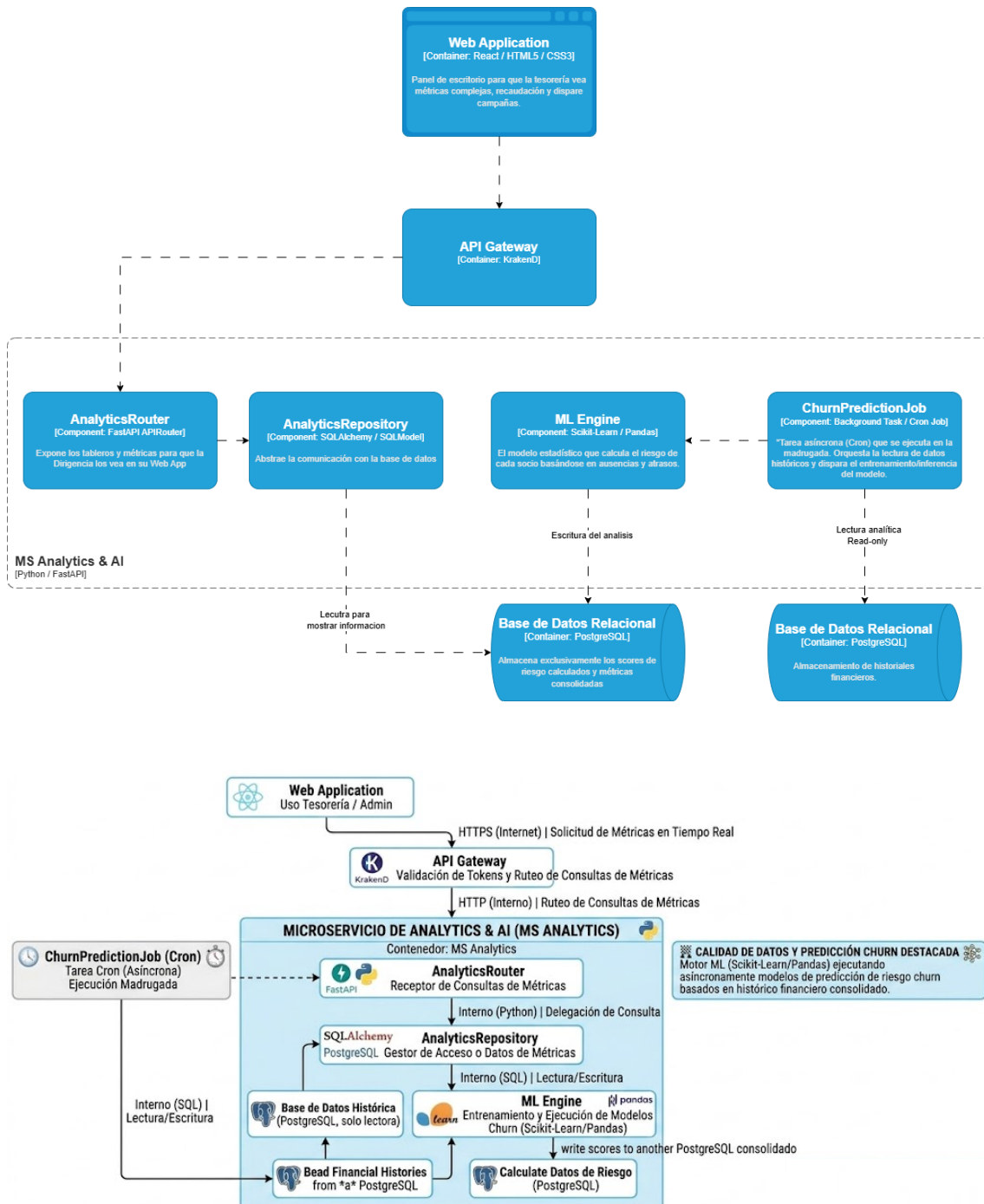


Figura 8: Diagrama C4 nivel 3: Componentes (Microservicio de analítica e inteligencia artificial).

Arriba: Vista técnica base.

Abajo: Vista visual simplificada. Tareas asíncronas y motor de Machine Learning.

1.3.7. Gateway

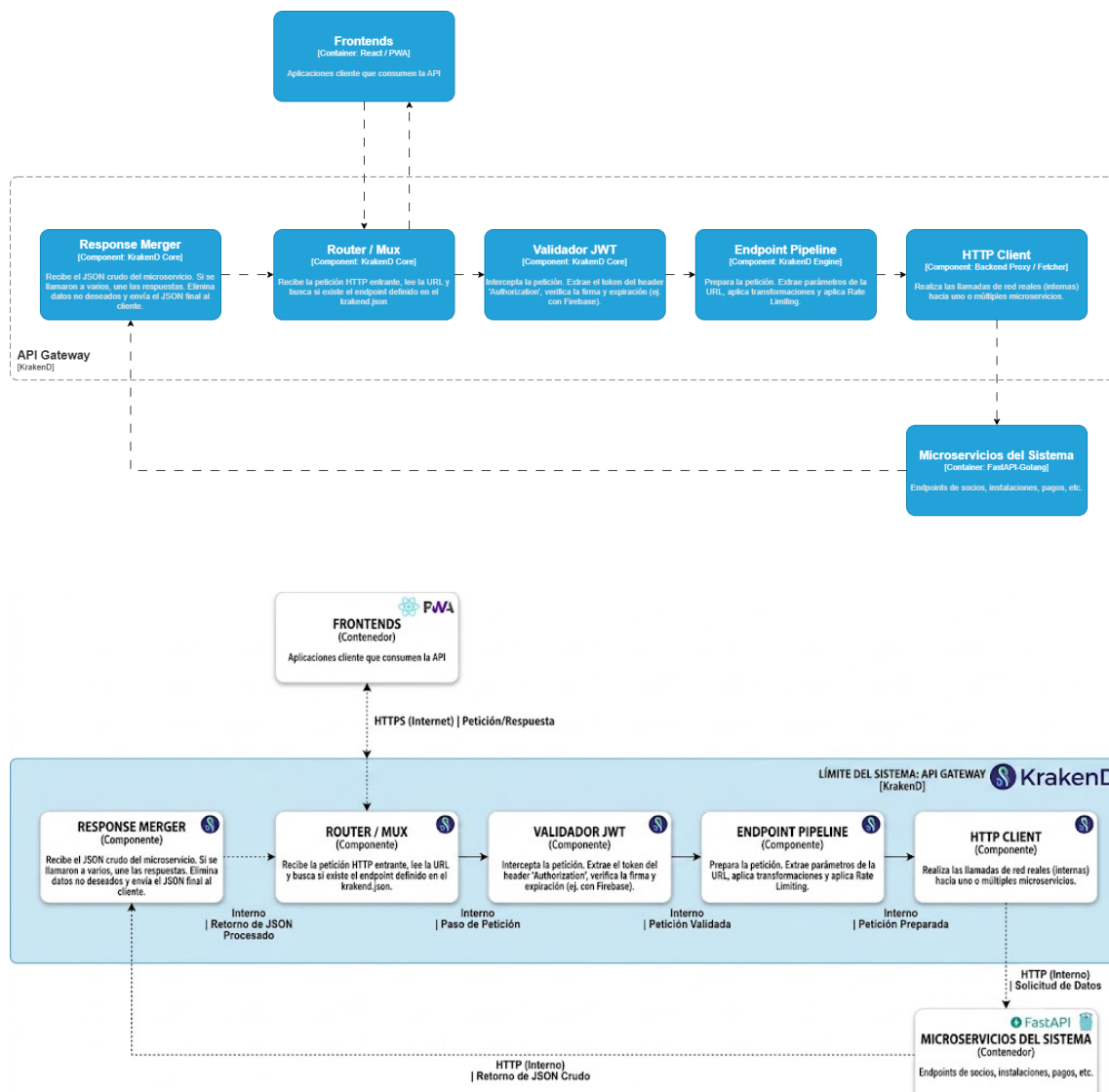


Figura 9: Diagrama C4 nivel 3: Componentes (API Gateway).

Arriba: Vista técnica base.

Abajo: Vista visual simplificada. Punto de entrada unificado basado en KrakenD, encargado de interceptar peticiones, validar tokens JWT, aplicar limitación de tasa (Rate Limiting) y enrutar el tráfico hacia los microservicios correspondientes.

1.3.8. Web administrativa (Panel de gestión)

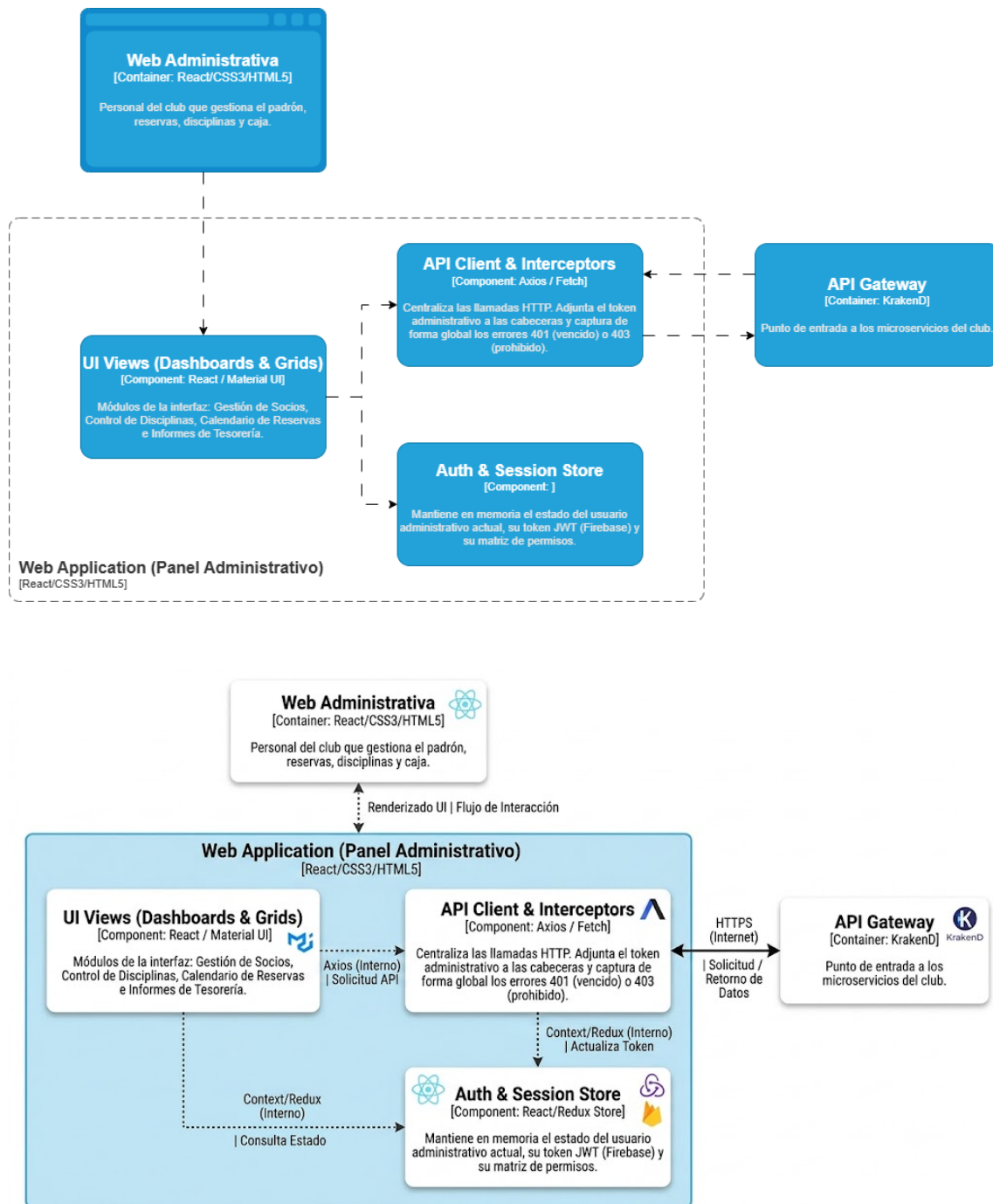


Figura 10: Diagrama C4 nivel 3: Componentes (Web administrativa).

Arriba: Vista técnica base.

Abajo: Vista visual simplificada. Aplicación frontend orientada al personal del club, que centraliza la administración de socios, disciplinas, reservas e informes, comunicándose de forma segura a través de la API Gateway.

1.3.9. App del Socio (PWA)

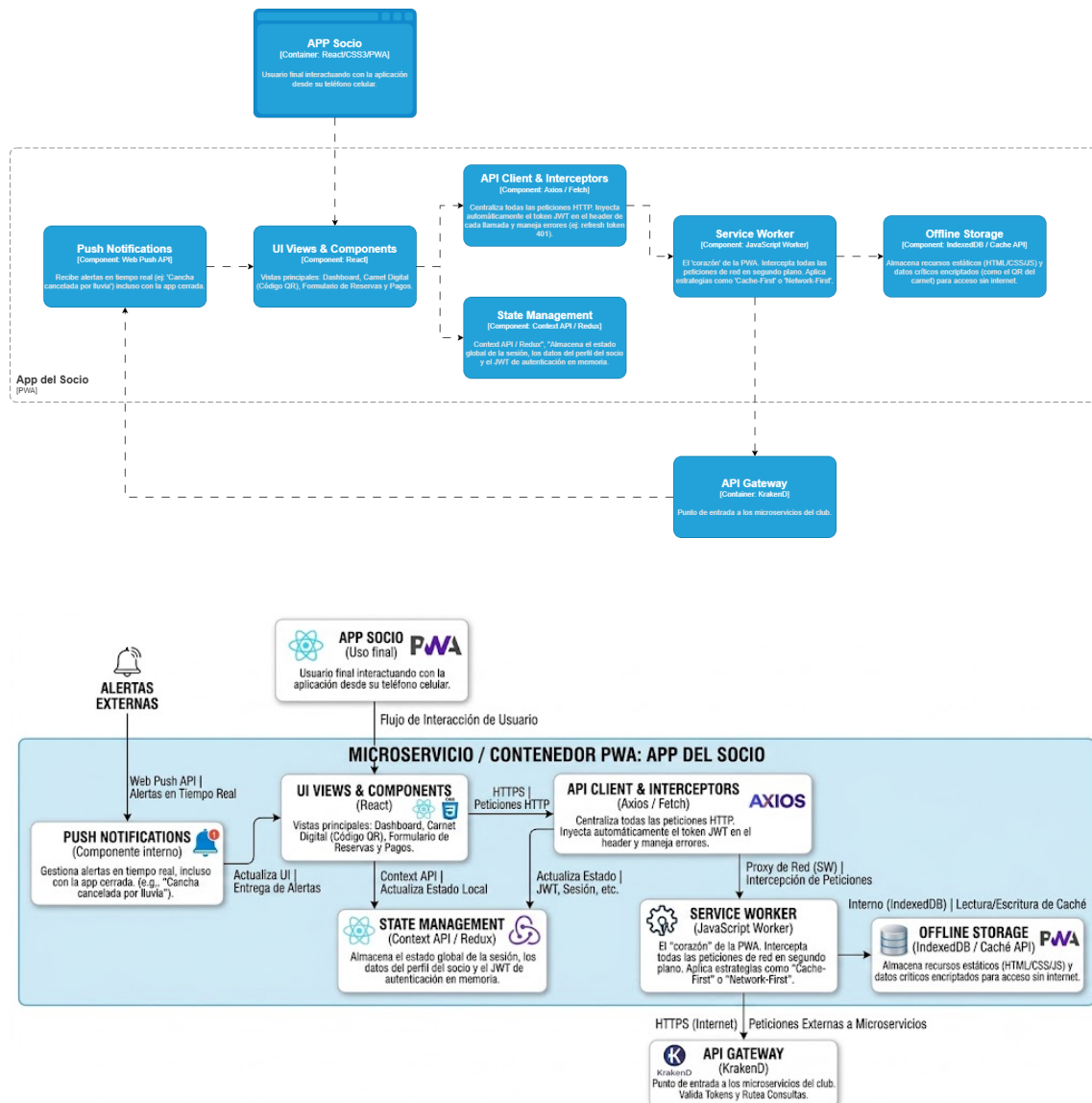


Figura 11: Diagrama C4 nivel 3: Componentes (App del Socio [PWA]).

Arriba: Vista técnica base.

Abajo: Vista visual simplificada. Aplicación Web Progresiva diseñada para el usuario final, con soporte offline, notificaciones push en tiempo real y almacenamiento local para garantizar el acceso rápido al carnet digital y gestión de reservas.

1.4. Monitoreo y Resiliencia: Servicio de Healthchecker

Para garantizar la alta disponibilidad, observabilidad y resiliencia del ecosistema de **Socio-Unido**, se ha incorporado un nodo dedicado exclusivamente al monitoreo continuo del sistema: el **Servicio de Healthchecker**.

Justificación Técnica y Arquitectónica

En arquitecturas distribuidas basadas en microservicios, la degradación o falla parcial de un componente no debe comprometer la integridad global de la plataforma ni pasar desapercibida. El patrón de *Health Check* implementado mediante un nodo observador independiente permite aislar la lógica de monitoreo del flujo principal de datos (como el API Gateway o los microservicios de negocio centrales).

El diseño y propósito de este nodo abarca los siguientes puntos críticos:

- **Monitoreo Activo (Pull-based):** Realiza rutinas programadas de verificación (pings HTTP/TCP) consultando los *endpoints* de salud designados (típicamente */health* o */metrics*) en cada uno de los microservicios y bases de datos.
- **Evaluación de Latencia y Estado de Salud:** Además de validar la correcta recepción de códigos HTTP 200 OK, el sistema actúa como una sonda que mide los tiempos de respuesta. Esto permite detectar bloqueos de base de datos, cuellos de botella en la red o degradación progresiva del rendimiento antes de que decante en una interrupción total del servicio (*Downtime*).
- **Auto-recuperación (Self-healing) y Orquestación de Alertas:** Al detectar una caída (cambio de estado de *UP* a *DOWN*), *timeouts* excesivos o anomalías estructurales, el motor de monitoreo está diseñado para actuar proactivamente. Puede ejecutar rutinas de mitigación e interactuar con la infraestructura para reiniciar contenedores comprometidos, o disparar alertas críticas mediante *webhooks* para notificar al equipo en tiempo real.

Desacoplar este componente es una decisión arquitectónica vital; asegura que bajo escenarios de alta concurrencia o ataques de denegación de servicio (DDoS) donde el API Gateway se encuentre comprometido o bajo estrés, el monitoreo transversal del sistema mantenga su capacidad operativa sin verse ralentizado por el tráfico de los usuarios finales.

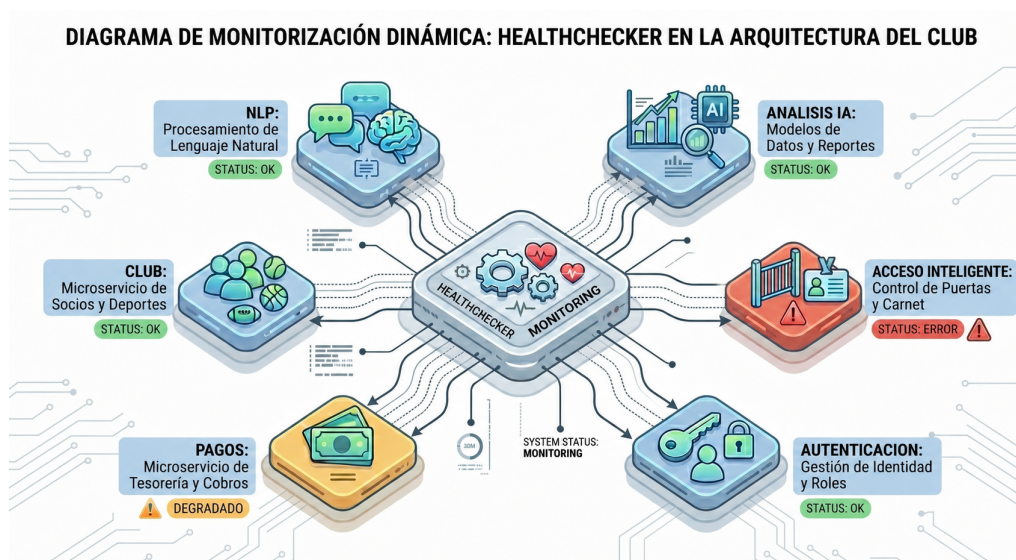


Figura 12: Diagrama ilustrativo de funcionamiento del HealthChecker.

1.4.1. Nivel 3 del modelo de arquitectura C4

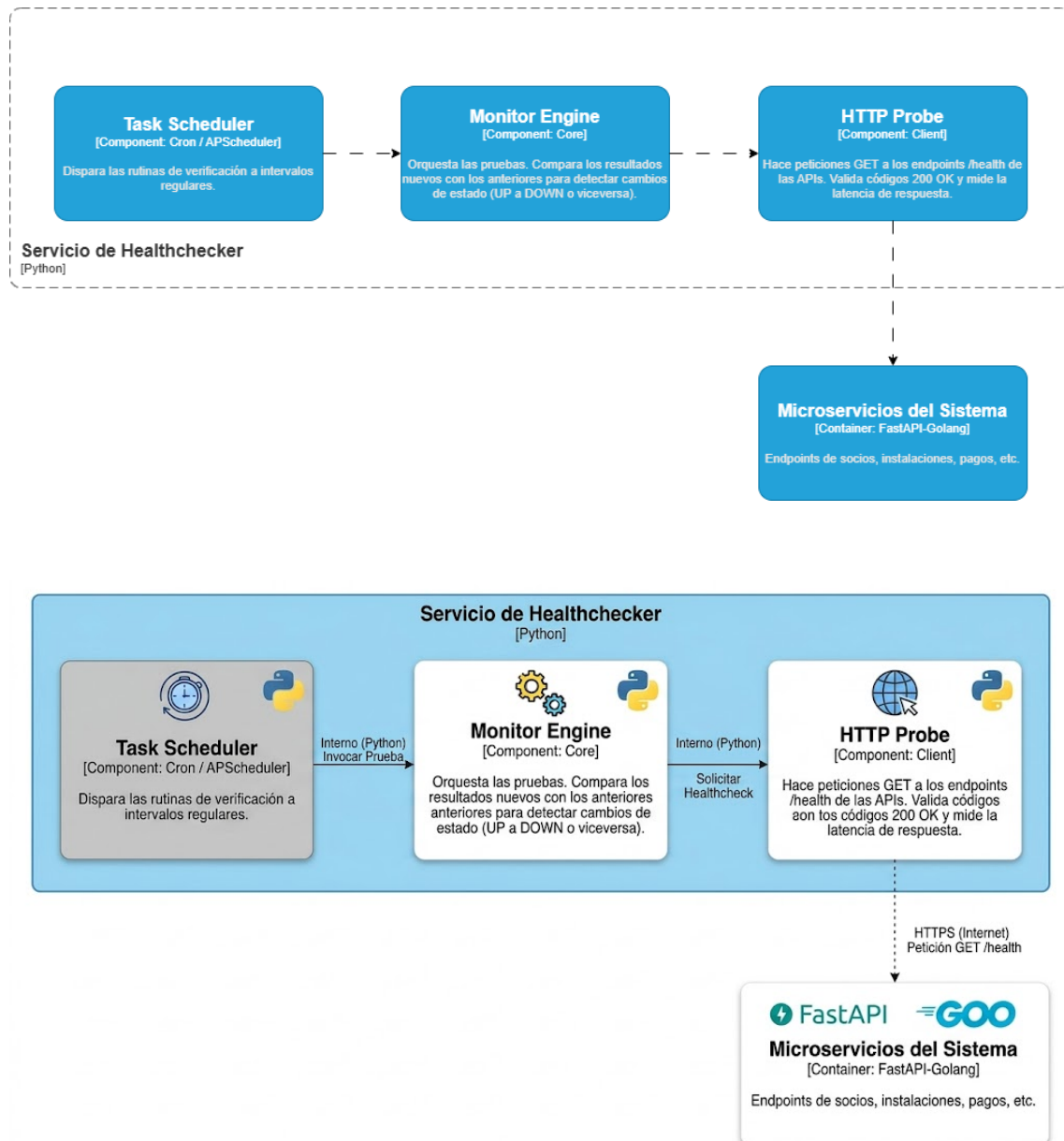


Figura 13: Diagrama C4 nivel 3: Componentes (Servicio de Healthchecker).

Arriba: Vista técnica base.

Abajo: Vista visual simplificada. Detalle interno del orquestador de tareas automatizadas (Cron), el motor lógico de monitoreo y las sondas de red.

1.5. Observabilidad y centralización de logs: Integración con Datadog

Complementando de forma sinérgica las capacidades de inspección reactiva y proactiva del **Servicio de Healthchecker**, la plataforma **SocioUnido** incorpora **Datadog** como pilar central de su estrategia de observabilidad corporativa. En una arquitectura orientada a microservicios donde las transacciones involucran la interacción secuencial de múltiples nodos descentralizados (tales como la pasarela de API, los servicios de autenticación, el microservicio central y las pasarelas de pago externas), la dispersión y el aislamiento de los registros de ejecución (*logs*) representan un desafío crítico para el diagnóstico de fallas y el análisis de rendimiento.

La adopción de Datadog resuelve esta problemática actuando como una plataforma unificada de agregación, correlación y análisis de datos en tiempo real, sirviendo como una herramienta indispensable en todas las etapas del ciclo de vida del software, con especial énfasis en las fases de desarrollo activo, pruebas integrales automatizadas y monitoreo continuo del ecosistema.

1.5.1. Ejes fundamentales de la implementación y casos de uso

El despliegue y la configuración de esta solución de telemetría transversal abarca las siguientes dimensiones técnico-arquitectónicas:

- **Agregación y centralización de logs heterogéneos:** Cada uno de los componentes del ecosistema de **SocioUnido** (los microservicios en sus respectivos contenedores, el API Gateway basado en KrakenD, y los motores de persistencia de datos) exporta sus flujos de salida estándar (`stdout` y `stderr`) de forma estructurada (típicamente en formato JSON). Datadog recolecta e indexa estos flujos de manera unificada, permitiendo al equipo de desarrollo realizar búsquedas transversales, filtrar por nivel de criticidad (`INFO`, `WARN`, `ERROR`) y reconstruir cronologías exactas de eventos sin necesidad de acceder de forma aislada a la infraestructura subyacente.
- **Trazabilidad distribuida y soporte para pruebas End-to-End (E2E):** Durante la ejecución de pruebas integrales o la simulación de flujos críticos del usuario final (por ejemplo, el login de un socio desde la PWA, pasando por la validación de identidad en el componente de autenticación hasta la confirmación de la pasarela de pagos), Datadog asocia un identificador único de traza (*Trace ID*) a la petición a través de mecanismos de APM (*Application Performance Monitoring*). Esta capacidad correlaciona los logs individuales de los distintos microservicios intervinientes en dicha transacción. Como resultado, es posible visualizar gráficamente el ciclo de vida completo de la solicitud, aislar cuellos de botella latentes y detectar con precisión milimétrica en qué punto exacto de la topología distribuida se originó una excepción.
- **Configuración de alertas proactivas para el ciclo de desarrollo:** Lejos de limitarse a un rol pasivo de auditoría *post-mortem*, la plataforma ha sido configurada con monitores inteligentes que evalúan de forma continua las métricas de ingesta. Mediante reglas basadas en lógica condicional, el sistema dispara alertas inmediatas ante anomalías tales como picos inusuales en la tasa de errores HTTP 5xx, excepciones no controladas en el microservicio de analítica e inteligencia artificial, o degradaciones de latencia en la lógica de validación por TOTP del microservicio de acceso inteligente. Estas notificaciones tempranas optimizan los tiempos de depuración en entornos de desarrollo y control de calidad (*QA*), previniendo el avance de regresiones hacia etapas posteriores del pipeline.
- **Visualización y análisis del comportamiento del sistema (*Dashboards*):** Se podrían diseñar tableros de control personalizados que consoliden las métricas operativas y de negocio más relevantes de la plataforma. Estos paneles interactivos ofrecen una representación visual intuitiva de la salud del sistema, permitiendo realizar análisis tendenciales de la carga de trabajo, observar los patrones de tráfico concurrentes y evaluar la estabilidad de las integraciones con webhooks externos (como la API de WhatsApp). Esta visibilidad continua facilita la toma de decisiones basada en datos para el refinamiento de la arquitectura y la planificación de la escalabilidad.